ATIVIDADE INTEGRADORA — JAVA E POO

Respostas resumidas | Aulas 01 a 08

Nome: Victor Manoel de Andrade Marrocos
Matrícula: 20262113058

Parte I — Questões teóricas

1. Classe é o modelo; objeto é uma instância da classe. Ex.: Curso é a classe; um curso de Java cadastrado é um objeto.

2. Atributos armazenam dados (estado); métodos definem ações (comportamento).

3. new cria um objeto na memória. Declarar uma referência cria uma variável que pode apontar para esse objeto.

4. Encapsulamento protege os dados e controla o acesso. Atributos públicos permitem alterações indevidas.

5. O construtor inicializa o objeto. this referencia o próprio objeto e diferencia seus atributos dos parâmetros.

6. Associação: objetos se relacionam. Agregação: relação todo-parte com partes independentes. Composição: as partes dependem do todo. Ex.: aluno e curso podem ser associados.

7. ArrayList é uma lista dinâmica; é preferível a um vetor quando a quantidade de elementos pode variar.

8. A superclasse é a classe-base; a subclasse herda dela. extends declara herança e super acessa membros ou construtor da superclasse.

9. Sobrecarga usa o mesmo nome com parâmetros diferentes. Sobrescrita redefine, na subclasse, um método herdado.

10. Polimorfismo permite tratar objetos diferentes por um tipo comum; uma referência da superclasse pode apontar para uma instância de subclasse.

11. Interface define um contrato. implements faz uma classe implementar uma interface; extends indica herança de classe ou extensão de interface.

12. Herança especializa uma classe; composição combina objetos. Composição é preferível quando se quer flexibilidade e menor dependência entre classes.

13. Baixo acoplamento reduz dependências. A injeção por construtor entrega as dependências prontas, sem a classe precisar criá-las.

14. SRP: uma responsabilidade por classe. OCP: permitir extensão sem modificar o código existente. DIP: depender de abstrações, não de implementações concretas.

15. Padrão de projeto é uma solução reutilizável para um problema recorrente. Deve ser aplicado quando necessário, pois o uso excessivo aumenta a complexidade.

16. Singleton garante uma única instância. O construtor privado impede criação externa e getInstancia() fornece a instância única.

17. Factory Method delega a criação de objetos a subclasses. O criador abstrato define o fluxo e os criadores concretos escolhem o produto.

18. Strategy encapsula cada regra de desconto em uma implementação comum, permitindo trocar entre desconto comum, estudantil ou promocional.

19. Após confirmar o pedido, o sistema publica um evento e os observadores de estoque, financeiro e auditoria recebem a notificação.

20. Adapter adapta uma interface incompatível, como converter reais em centavos. Facade oferece uma operação simples que coordena várias partes do sistema.

Parte II — Perguntas de análise da implementação

1. A classe Configuracao. A instância é controlada pelo atributo static final INSTANCIA, pelo construtor privado e por getInstancia().

2. O encapsulamento aparece nos atributos private; a validação ocorre nos construtores de Aluno e Curso.

3. Aluno e Curso têm associação: o aluno mantém uma lista de cursos, que podem existir independentemente dele.

4. ArrayList é usado para armazenar os cursos do aluno e os observadores da matrícula.

5. Desconto define o contrato aplicar(double valor), permitindo diferentes políticas de desconto.

6. Criar uma nova classe que implemente Desconto e aplique 20%, sem alterar ServicoMatricula.

7. ServicoMatricula produz o evento após a matrícula; AvisoEmail e Auditoria recebem as notificações.

8. Porque o sistema legado recebe valores em centavos inteiros, enquanto a aplicação trabalha com reais.

9. O método criar(), sobrescrito por CriadorTexto e CriadorResumido.

10. FachadaAcademica coordena a matrícula e a emissão do comprovante, deixando Main mais simples.